# Miniproject 1: Classification of Handwritten Digits

Davoud Mirzaei

Classification of handwritten digits is a standard problem in pattern recognition. A typical application is the automatic reading of zip codes on envelopes. This miniproject aims to practice a classification technique using SVD.

The problem is as follows: *Given a set of manually classified digits as a training set, classify a set of unknown digits as the test set.*

In Figure 1 a sample of handwritten digits $0, 1, \ldots, 9$ is illustrated. The images are downloaded from `https://www.nist.gov/itl/products-and-services/emnist-dataset`. Each image is a $28 \times 28$ grayscale image but we stack the columns of the image above each other so that each image is represented by a vector in $\mathbb{R}^{784}$.

Figure 1: A sample of handwritten digits $0, 1, \ldots, 9$.

The training set of each kind of digits $0, 1, \ldots, 9$ can be considered as a matrix $A \in \mathbb{R}^{m \times n}$, with $m = 784$ and $n =$ 'the number of training digits of one kind'. So, we have a total of 10 training matrices, each corresponding to one of the digits $0, 1, \ldots, 9$. Assume that $A$ is one of them. The columns of $A$ span a linear subspace of $\mathbb{R}^{784}$. However, this subspace cannot be expected to have a large dimension, because columns of $A$ represent the same digit with different handwritings. One can use SVD $A = U\Sigma V^T$ to obtain an orthogonal basis for the column space of $A$ (or range$(A)$) via the columns of factor $U$. Then any test image of that kind can be well represented in terms of the orthogonal basis $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_m\}$. Let us describe it in more detail. We have

$$A = U\Sigma V^T = \sum_{j=1}^{m} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^T,$$

thus the $\ell$-th column of $A$ (the $\ell$-th training digit) can be represented as

$$\boldsymbol{a}_\ell = \sum_{j=1}^{m} (\sigma_j v_{j\ell}) \boldsymbol{u}_j$$

which means that the coordinates of image $\boldsymbol{a}_\ell$ in terms of orthogonal basis $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m\}$ are $\sigma_j v_{j\ell}$. This means, in another point of view, that solving the least squares problem

$$\min_{\boldsymbol{x}} \|U\boldsymbol{x} - \boldsymbol{a}_\ell\|_2$$

results in an optimal vector $\boldsymbol{x} = [\sigma_1 v_{1\ell}, \ldots, \sigma_m v_{m\ell}]^T$ with residual 0. As we pointed out, most columns of $A$ are nearly linearly dependent as they represent different handwritten for the same digit. If we translate it to columns of $U$, this means that a few leading columns of $U$ should well capture the subspace. For this reason the orthogonal vectors $\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots$ are called *singular images*. See Task 2 below. Consequently, we can use a $k$-rank approximation of $A$ where $k \ll m$. In this case each $\boldsymbol{a}_\ell$ can be well represented by orthogonal basis $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$ in a least squares sense with a small residual.

Now, let $\boldsymbol{d} \in \mathbb{R}^m$ be an image of a digit outside the training set (which is called a *test digit*). It is reasonable to assume that $\boldsymbol{d}$ can be well represented in terms of singular images $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$ of its own kind with a relatively small residual. We must compute how well $\boldsymbol{d}$ can be represented in the 10 different bases, each corresponding to one of the digits $0, 1, \ldots, 9$. This can be done by computing the residual vector in the least squares problems of the type

$$\min_{\boldsymbol{x}} \|U_k\boldsymbol{x} - \boldsymbol{d}\|_2 \tag{0.1}$$

where $\boldsymbol{d}$ represents a test digit and $U_k \in \mathbb{R}^{m\times k}$ consists of the first $k$ columns of $U$.

Altogether, a SVD-based classification algorithm of handwritten digits consists of two steps:

- **Step 1 (training):** For the training set of known digits, compute the SVD of each set of digits of one kind (Compute ten SVDs for digits $0, 1, \ldots, 9$).
- **Step 2 (classification):** For a given test digit $\boldsymbol{d}$, compute its relative residual in all ten bases using (0.2). If a residual in a class is smaller than all the others, classify $\boldsymbol{d}$ in that class.

To get started with the miniproject, download the *training* set `TrainDigits.npy` consisting of 240000 images (24000 images per any digit), and the *test* set `TestDigits.npy` consisting of 40000 images (4000 images per digit) as a zip document from the Miniproject page in Studium. The document contains *label* files `TrainLabels.npy` and `TestLabels.npy` as well, which are manual classifications of training and test sets, respectively. The training labels will be used to learn and the test labels to verify your algorithm.

Note: In tasks 2 and 3 below, for each digit consider using only the initial 400 images out of the total 24000 training images.

- **Task 1:** Show that the solution of least square problem (0.1) is $\boldsymbol{x} = U_k^T \boldsymbol{d}$ and the residual is

$$residual = \|(I - U_k^T U_k)\boldsymbol{d}\|_2. \tag{0.2}$$

  Hint: Use the orthogonality of columns of $U$.

- **Task 2:** For training matrices of digits 3 and 8 (each of size $784 \times 400$), compute the SVD and plot the singular values (two plots). Report your observation and try to justify it. Also plot the first three singular images $\boldsymbol{u}_1, \boldsymbol{u}_2, \boldsymbol{u}_3$ for digits 3 and 8, separately (six plots), and report your observation.

- **Task 3:** Implement the above SVD-based classification algorithm in a Python code. Use only $n = 400$ training images for each digit (ten matrices each of size $784 \times 400$). However, test the algorithm on all 40000 test images and compare your classifications with the exact labels `TestLabels.npy`. Use different values $k = 5, 6, \ldots, 20$ (number of basis functions), and report the percentage of the success of this SVD-based algorithm for each digit.

- **Task 4:** Write a report for your mini-project, utilizing either Word or LaTex. Please follow the structure outlined on the page "About Miniprojects" in Studium. The report should contain the solution to Task 1, the Python code snippets, and figures, along with discussions, and your comments on your observations. Provide proper captions and labels for figures.

To load images you can write, e.g., `TrainDigits = numpy.load('TrainDigits.npy')`. Be sure that .npy files are copied in the directory. To plot the digits you need to reshape the columns into $28 \times 28$ matrices using `numpy.reshape` and then use `imshow` command to plot the digit. For example:

```
import numpy as np
import matplotlib.pyplot as plt

TrainDigits = np.load('TrainDigits.npy')
d = TrainDigits[:,0]            # The first digit in the training set
D = np.reshape(d, (28, 28)).T   # Reshaping a vector to a matrix
plt.imshow(D, cmap ='gray')     # Plot of the digit
```

See lecture presentations and Lab exercises for Python syntaxes for vector and matrix manipulations and SVD.